ASK KIQSTAT · COST OPTIMISATION & FINANCIAL SAFETY · 29 OCTOBER 2026

Making Ask KiqStat pay its way: what we found and what is ready to test

The old match write-ups, not Ask, used up the Anthropic credit. Cost logging, hard spending limits and a benchmark lab are built and pushed to `dev`, all switched to observe-only so production behaves exactly as before. The one step not done is the live benchmark: it needs a funded test key, which this environment doesn't have. Real cost-per-question figures come from that run.

MEASURED from the database, production logs, the code or a test run

ESTIMATED from prompt sizes and published prices; replaced by measurements once the lab runs

CREDIT 7,960 match write-ups ran in about 30 hours, from 9 Oct 18:19 to 11 Oct 00:23 UTC, against about 93 Ask questions in the same window. The write-ups stopped when the credit ran out. **MEASURED**

STOPPED Nothing has called the write-up generator since 18 October, and its code was deleted on 25 October. Ask is now the only code that calls Anthropic. **MEASURED**

BUILT Every model call's tokens and cost are logged, there are per-question and per-account spending limits, and alerts and a lab compare five settings over 40 test questions. **MEASURED** by 1,077 passing tests and a 13-check concurrency test.

NOT RUN The benchmark needs `ANTHROPIC_TEST_API_KEY`. Until it runs, every cost per question in this report is an estimate.

MARGIN With the proposed daily caps, AI cost per subscriber can't pass about 40% of the monthly price (under 50% on yearly plans), however much they ask. **ESTIMATED** at ₦1,600 to the dollar.

1. Separating Ask's costs from the write-ups

DAY (UTC)	WRITE-UPS GENERATED	ASK QUESTIONS ANSWERED (SIGNED IN)
9 Oct (from 18:19)	1,024	6
10 Oct	6,847	13
11 Oct (to 00:23)	89	0
12 Oct onwards	0	0

Source: `match_analyses` and `ask_usage` row counts MEASURED. Visitors without an account also had 74 questions answered over the period. Their counts have no date, so they are left out of the table.

- **Can't be split by API key or workspace.** Both features used the same key (`ANTHROPIC_API_KEY`) and the same model (Claude Haiku 4.5), so Anthropic's console can't tell them apart. MEASURED from the code at the time (`src/lib/ai/analyst.ts`, removed in commit e454462).
- **Split by volume instead:** about 7,960 write-up calls against about 280 Ask calls (93 questions at about 3 calls each). Weighting by typical tokens, the write-ups were about **95% of the spend**. ESTIMATED. The Anthropic usage page for 9 to 11 October can confirm the total.
- **Going forward:** Ask reads `ANTHROPIC_ASK_API_KEY` when it is set. Create it in its own Anthropic workspace, called "Ask KiqStat", with its own monthly spend limit, so the console shows Ask on its own. Put the lab's key in a second workspace, "Ask lab".

2. Usage and cost logging

- **Every model call:** uncached input, cache-write, cache-read and output tokens (thinking included), cost, stop reason, time, and whether it was cancelled or re-issued. table ask_calls
- **Every question:** plan (guest, free, Pro, VIP, pass), number of calls, total tokens and cost, outcome (answered, stopped by the budget, cancelled, error, refused, blocked), the settings used, and what the guard did or would have done. No question or answer text is stored for live users. table ask_questions
- **Cost** comes from each response's own usage block at list prices: Haiku 4.5 is $0.10 / $0.50 per million tokens, cache writes 1.25×, cache reads 0.1×, and 5× above 100K-token prompts. Unit-tested. src/lib/ask/cost.ts
- **Reporting by plan** is on the Ask lab page: questions, answered share, average and highest cost, the most any one account spent in a day, and the total. Accounts are counted but never named.

3. Token use: what will be tested

The lab runs the same 40 questions under each preset, on real fixtures from this week. The questions cover 8 simple, 6 analysis, 6 combo, 6 tool-heavy, 6 price, 3 live and 5 off-topic or edge cases.

PRESET	OUTPUT TOKENS PER CALL	CALLS	EFFORT	EXPECTED EFFECT
As in production	8,000	6	medium	Baseline.
Shorter ceiling	1,500	6	medium	Typical cost unchanged (answers use a few hundred tokens); worst question about ₦53 → about ₦19. ESTIMATED
Shorter, 4 calls	1,500	4	medium	Worst question about ₦14. Watch combo and tool-heavy answers for missing lookups. ESTIMATED

Shorter, 4 calls, low effort	1,500	4	low	Less thinking billed on every call; often holds for chat. Quality must be checked. ESTIMATED
With the ceiling on	1,500	4	medium	Same, with $0.01 (₦16) per question enforced: shows how often the ceiling bites and what users see.

Kept as they are: prompt caching (the fixed instructions and tools, plus the conversation within a question) and the compact tool data, both unchanged in the new engine.

4. Financial safeguards

CONTROL	HOW IT WORKS	DEFAULT
Per-question ceiling	Before each call, the most it could cost is worked out (its whole prompt at the cache-write rate plus every output token allowed). If that would pass the ceiling, the call becomes a short final answer with no more lookups, or the question stops.	`$0.02`
Per-account daily cap	Each question reserves its ceiling against the account's day before the first call, then settles to what it really cost. Near the cap it is granted whatever is left, if that covers a short answer, and runs under that smaller ceiling.	Free $0.01 · Pro $0.04 · VIP $0.10 · pass $0.12 · guest $0.005
Ask's daily budget	The same reservation, against all of Ask. When the budget is reached, questions pause until midnight Lagos time and an alert goes out.	`$5 a day`
Cancellations, retries, parallel requests	Money is booked at settlement whatever happened. Cancelled and failed calls are costed from their partial usage. The question count is still handed back when there is no answer, but the money isn't.	Always on
Alerts by email	Billing or key failure (once an hour); 5 or more failed questions in 15 minutes; daily budget at 50%, 80% and 100%.	To `ASK_ALERT_EMAIL` or the support inbox
Safe fallback	A billing failure says "unavailable" (not "busy", which made people retry). A stopped question says to ask about one match at a time. Neither counts against the user.	Always on
Guard mode	`observe` reserves, logs and alerts but never blocks. `enforce` blocks.	`observe`

Settings: `ASK_GUARD`, `ASK_REQUEST_CEILING_USD`, `ASK_DAILY_CAP_USD_<PLAN>`, `ASK_DAILY_BUDGET_USD`, `ASK_MAX_TOKENS`, `ASK_MAX_ROUNDS`, `ASK_EFFORT`, `ASK_PASS_DAILY`. Each defaults to how production runs today.

Pass allowance and wording

- **Pass allowance:** `ASK_PASS_DAILY` sets questions a day on a pass. Unset, a pass keeps its plan's allowance (VIP 100), as today. The proposal is 30; it needs your approval before it goes to production.
- **Wording, fixed:** "Pro: unlimited" and "Pro makes it unlimited" now say "Pro: 30 a day", and paid members see "X of 30 questions left today". The same fix is made in Forge: "Pro: 100 a day". This changes wording only, not allowances.

5. Evidence that the controls work

Spending limits under parallel load MEASURED

The real SQL from migration 0088, run on a local Postgres 16 with genuinely parallel connections (`scripts/ask-spend-concurrency.sh` ; nothing touched Supabase):

CHECK	RESULT
50 questions at once, room for 10	10 allowed, 40 refused
After 10 settled at $0.003 each: spent, still reserved	$0.030, $0
Second wave of 50, room for 7	7 allowed
Near the cap, the room left is granted, never more	reserved = cap exactly
Less than one short answer left	refused
Observe mode: blocks nothing, reports going over	20 of 20 through, flagged
Cancelled question booked at its real cost	$0.001234 booked
Browser roles calling the functions or reading the tables	refused

This test caught a real bug before it shipped: a column-name clash in the reservation function would have made every reservation fail. It is fixed, and the script now passes all 13 checks.

Per-question ceiling MEASURED

Unit tests run the engine against a fake model that always looks things up and always writes the maximum. Across 200 random settings and ceilings, no question went over its ceiling. Cancelled calls are costed, a credit failure is flagged as billing, and observe mode leaves behaviour unchanged. src/lib/ask/engine.test.ts

6. What the caps guarantee

The most AI can cost one subscriber in a month is the daily cap × 30, whatever they ask. Against each plan's price, before Paystack's fee: ESTIMATED at ₦1,600 to the dollar.

PLAN	DAILY CAP	MOST IN 30 DAYS	MONTHLY PRICE	SHARE	SHARE ON YEARLY
Pro	$0.04	₦1,920	₦5,000	38%	**48% (₦4,000/mo)**
VIP	$0.10	₦4,800	₦12,000	40%	**50% (₦9,600/mo)**
Pass (one day)	$0.12	₦192	₦500	38%	–
Free	$0.01	₦480	₦0	covered by the $5 daily budget	–

The trade-off to settle with the lab results: at about ₦2 a question, a VIP member asking all 100 questions would spend about $0.125 a day and reach the $0.10 cap at around question 80. A Pro member at 30 typical questions sits just under the $0.04 cap. Heavy questions reach the cap sooner. The lab's measured average and 95th-percentile costs decide whether to raise the caps (thinner margin), lower the allowances, or accept that the cap rarely bites in practice. `ESTIMATED`

7. Remaining risks

- **Cost per question isn't measured yet.** Every figure above is estimated until the lab runs on a funded test key.
- **The ceiling relies on a token estimate.** The first call's prompt size is estimated on the high side (3,200 fixed tokens, 2.5 characters per token). If it were ever low, one question could go slightly past its ceiling. The next reservation then sees the real spend, so an account can only go over its cap by part of one question.
- **Prices are hard-coded** in `src/lib/ask/cost.ts`. A price change at Anthropic needs a one-line update there, or logged costs drift.
- **The exchange rate** is assumed at ₦1,600. Pass `ASK_NAIRA_PER_USD` to the lab page for the current rate.
- **Yearly plans** have the thinnest margin (up to 50%). Stress-test them when setting final caps.
- **Migration 0088 isn't applied.** Until it is, logging and caps quietly do nothing (observe mode lets questions through). Enforce mode refuses questions when it can't reach its tables.
- **Product quality** is unchanged in production while the guard observes. Lower ceilings, fewer calls or low effort need the lab's side-by-side ratings before going live.

Next steps

1. **Apply migration 0088** (`supabase/migrations/0088_ask_cost_tracking.sql`). It only adds tables and functions and changes nothing existing.
2. **Create two Anthropic workspaces with spend limits**, "Ask KiqStat" and "Ask lab". Put the lab key in Vercel as `ANTHROPIC_TEST_API_KEY` for **Preview only**. Top up the production credit.
3. **Run the lab on the `dev` preview** (Admin → Product → Ask lab): all five presets, about $1 in total. Rate the answers side by side.
4. **I'll write up the measured results**: real cost per question, highest and worst case, latency, quality ratings and the measured margin per plan and billing period. Then we set the caps and allowances together.
5. **After approval:** set `ANTHROPIC_ASK_API_KEY` and the chosen settings in Production, and move `ASK_GUARD` from observe to enforce after a week of observed data.

Files

- `src/lib/ask/engine.ts · cost.ts · config.ts · ledger.ts · lab.ts · bench-cases.ts`
- `src/app/api/ask/route.ts`
- `src/app/admin/(protected)/ask-lab/page.tsx · src/app/actions/admin/ask-lab.ts`
- `supabase/migrations/0088_ask_cost_tracking.sql`
- `scripts/ask-spend-concurrency.sh`

No API keys, credentials or customer details are in this report. Production pricing, allowances and partner terms are unchanged.